

Appendix C

Definition of the Programming Language CPRL

Introduction

CPRL (for **C**ompiler **P**roject **L**anguage) is a small but complete programming language with constructs similar to those found in C, Java, and Ada. CPRL was designed to be suitable for use as a project language in an advanced undergraduate or beginning graduate course on compiler design and construction. Its features illustrate many of the basic techniques and challenges associated with language translation.

C.1 Lexical Considerations

General

CPRL is case sensitive. Upper-case letters and lower-case letters are considered to be distinct in all tokens, including reserved words.

Whitespace characters (space character, tab character, and end-of-line) serve to separate tokens; otherwise they are ignored. No token can extend past an end-of-line. Spaces may not appear in any token except character and string literals.

A comment begins with two forward slashes (//) and extends to the end of the line (or end of the file, whichever occurs first).

```
temp := x;    // swap values of x and y
x := y;
y := temp;
```

CPRL does not define a maximum line length for source code files.

Identifiers

Identifiers start with a letter and contain letters and digits. An identifier must fit on a single line, and all characters of an identifier are significant.

```
identifier = letter { letter | digit } .
letter = 'A'..'Z' + 'a'..'z' .
    // letter = [A-Za-z]    equivalent regular expression character class
digit = '0'..'9' .
    // digit = [0-9]        equivalent regular expression character class
```

Reserved Words

The following 37 identifiers are keywords in CPRL, and they are all reserved.

Boolean	Byte	Char	Integer	String	and	array
class	const	else	exit	false	for	fun
if	in	loop	mod	not	of	or
private	proc	protected	public	read	readln	record
return	then	true	type	var	when	while
write	writeln					

Note that some keywords such as `class`, `private`, etc. are not currently used in CPRL but are reserved for possible future use. Such keywords are essentially terminal symbols in the context-free grammar that do not appear in any rule.

Literals

An integer literal can be defined using decimal, hexadecimal, or binary notation.

```
intLiteral = decimalLiteral | hexLiteral | binaryLiteral .
decimalLiteral = digit { digit } .
hexLiteral = ( "0x" | "0X" ) hexDigit { hexDigit } .
binaryLiteral = ( "0b" | "0B" ) binaryDigit { binaryDigit } .
```

In the above rules, `binaryDigit` and `hexDigit` have the usual definitions.

```
binaryDigit = '0'..'1' .
hexDigit = '0'..'9' + 'A'..'F' + 'a'..'f' .
```

A Boolean literal is either “true” or “false”, and both of these words are reserved.

Similar to most C-based languages, CPRL uses the backslash (`\`) as a prefix character to denote escape sequences within character and string literals. The escape sequences used by CPRL are similar to those used in other languages.

<code>\t</code>	tab
<code>\n</code>	linefeed (a.k.a. newline)
<code>\r</code>	carriage return
<code>\"</code>	double quote
<code>\'</code>	single quote (a.k.a. apostrophe)
<code>\\</code>	backslash

A character literal is a single printable character or an escaped character enclosed by a pair of apostrophes (sometimes called single quotes). Examples include `'A'`, `'x'`, and `'\''`. A character literal is distinct from a string literal with length one. A backslash is required for

a character literal containing an apostrophe (single quote). Thus, `'\''` can be used to represent a character literal consisting of a single quote, but `''` is not valid.

For convenience, we define a printable character as Unicode character encodable in 16 bits (using UTF-16) that is not an ISO control character, although there are some 16-bit Unicode characters that are not ISO control characters that are also not printable. The Java method `isISOControl()` in class `Character` can be used to test this condition.

A string literal is a sequence of zero or more printable or escaped characters enclosed by a pair of quotation marks (double quotes). Analogous to character literals, backslashes are required whenever a string literal contains quotation marks. Examples of string literals include `"Hello, world."`, `"It's Friday!\n"`, and `"He said, \"Good morning.\""`.

Other Symbols

The following symbols serve as delimiters, operators, and special symbols in CPRL.

```
// arithmetic, bitwise, and shift operators
+   -   *   /   &   |   ^   ~   <<   >>

// relational operators
=   !=   <   <=   >   >=

// assignment, grouping, and other symbols
:=   (   )   [   ]   {   }   ,   :   ;   .   ..   ?

// special scanning symbols
EOF, unknown
```

Note that CPRL uses the reserved word “mod” instead of the symbol `%` as the modulus or remainder operator. Similarly, CPRL uses the reserved word “not” instead of the symbol `!` for logical negation. (But the “not equal” relational operator is `!=`.)

C.2 Types

CPRL is a statically-typed language.

- Every variable, constant, or expression in the language belongs to exactly one type.
- Type is a static property and can be determined by the compiler.

In general, static typing allows better error detection at compile time as well as generation of more efficient object code.

Scalar Types

There are four predefined scalar types in CPRL – Boolean, Byte, Integer, and Char. In addition, CPRL has a user-defined scalar type called an enumerated type (or simply an enum type).

Type Boolean

Type Boolean is treated as a predefined scalar type with two values, `false` and `true`. It is essentially equivalent to type `boolean` in Java, `Boolean` in Kotlin, and type `bool` in C++.

Type Byte

Type Byte is an **unsigned** 8-bit numeric type with values in the range 0..255. Values of type Byte are promoted to type Integer when used as expressions.

Type Integer

Type Integer is a numeric type that is equivalent to type `int` in Java or `Int` in Kotlin.

Type Char

Type Char is a predefined character type that is roughly equivalent to type `char` in Java except that all characters in CPRL must be encodable in 2 bytes (16 bits) using UTF-16. This restriction greatly simplifies the treatment of Char values in CPRL since they all occupy exactly 2 bytes.

Enumerated Types

An enumerated type is defined by listing the identifiers that are the actual values of the type enclosed in angle brackets. Each enumerated identifier defines a constant value whose name is scoped to the enum type.

Example

```
type Day = <Sun, Mon, Tue, Wed, Thu, Fri, Sat>;
```

An enumerated type is distinct from all other types. In particular, while the enumerated constants have some properties similar to those of type Integer, an enumerated type is distinct from type Integer.

Composite Types

There are three composite types in CPRL – array types, record types, and the predefined type `String`. Array types and record types must be declared before the type name can be referenced, but array variables can be created without first declaring a type, as in

```
var a : array[10] of Integer;
```

Array Types

CPRL supports one dimensional array types, but arrays of arrays can be declared. An array type is defined by giving the number of elements in the array and the component (element) type.

Examples

```
type T1 = array[10] of Boolean;
type T2 = array[10] of Integer;
type T3 = array[10] of T2;
```

If n is the number of array elements, then array indices are integers ranging from 0 to $n-1$ or enumerated constants with similar underlying range values. No bounds checking is performed when accessing an element of an array.

Type String

String is a predefined composite type in CPRL. String variables can be initialized using string literals or declared string constants. For a string variable, the length is determined when it is assigned a value, either via initialization, an assignment statement, or a read statement.

Examples

```
var name : String := "Davis";
var month : String;
```

Similar to arrays, the individual characters of a string can be accessed using the array index notation. Valid indices range from 0 to $n-1$, where n is the length of the string. No bounds checking is performed when accessing a character in a string. Similar to records, the length of a string is accessed using the dot notation, as in `name.length`.

Record Types

CPRL supports record types that are similar to structs in C and C#. A record type is defined by specifying the fields in the record. Unlike arrays, whose elements all have the same type, the fields of a record can have different types.

Examples

```
type Point = record
{
  x : Integer;
  y : Integer;
};

type Month = record
{
  name      : String;
  maxDays   : Integer;
};
```

The fields of a record are accessed using the dot notation, as in `p.x` or `m.name`.

C.3 Constants and Variables

Constants and variables must be declared before they can be referenced.

Constants

A constant (a.k.a. named constant or manifest constant) is essentially a name for a literal value. Constants are introduced by declarations that conform to the following rule.

```
"const" constId "[:=" [ "-" ] literal ";" .
```

The type of the constant identifier is determined by the type of the literal, which must be an integer literal, a character literal, a boolean literal, or a string literal. The optional minus sign (" -") is allowed only for integer literals.

Example

```
const arraySize := 100;
const errorSig  := -1;
```

Variables

Variables are introduced by declarations of the following form.

```
var varId1, varId2, ..., varIdn : typeName := initializer;
```

The type name can be one of the predefined scalar types such as Boolean or Integer, or it can be an identifier representing an array type, and enumerated type, or a record type. A variable declaration can contain an optional initialization value, which must have the same type as the variables being declared.

For array variables, the type name does not have to be an identifier; an array type constructor can be used. So, for example, the following is also allowed.

```
var x : array[100] of Integer;
```

Initializers for scalar types and string types are simply constant values; i.e., literals or declared constants. But array types and record types can have composite initializers, which are enclosed in braces and separated by commas. The number of initializers within the braces must equal the number of components in the composite type. Composite initializers can be nested; e.g., to initialize an array of records or an array of arrays.

Here are the grammar rules for initializers.

```
initializer = constValue | compositeInitializer .
constValue = ( [ "-" ] literal ) | constId .
compositeInitializer = "{" initializer { "," initializer } "}" .
```

Examples

```

var x1, x2 : Integer;           // x1 and x2 are not initialized
var x3, x4 : Integer := 1;     // both x3 and x4 are initialized to 1
var found  : Boolean := false;
var name   : String  := "Jack";
var a : array[6] of Integer := { 0x00, 0x01, 0x02, 0xFD, 0xFE, 0xFF };

type DaysPerMonth = array[13] of Integer;
var maxDays : DaysPerMonth := { 0, 31, 29, 31, 30, 31, 30,
                                31, 31, 30, 31, 30, 31 };

const numRows := 3;
const numCols := 3;
type Row      = array[numCols] of Integer;
type Matrix   = array[numRows] of Row;   // a 3x3 matrix of integers
var a : Matrix := {
    { 2, 7, -3 },
    { 1, 1,  5 },
    { 7, 8,  9 }
};

var i : Matrix := {                                     // the identity matrix
    { 1, 0, 0 },
    { 0, 1, 0 },
    { 0, 0, 1 }
};

type Month = record
{
    name      : String;
    maxDays   : Integer;
};
type Months = array[13] of Month;
var months : Months := { { "invalid",    0 },
    { "January",    31 }, { "February",  29 },
    { "March",      31 }, { "April",     30 },
    { "May",        31 }, { "June",      30 },
    { "July",       31 }, { "August",    31 },
    { "September", 30 }, { "October",   31 },
    { "November",  30 }, { "December",  31 }
};

```

C.4 Operators and Expressions

Operators

The operators, in order of precedence, are as follows.

- | | | | | | | | |
|----------------------------------|-----|----|-----|----|----|----|----------------------|
| 1. Boolean/bitwise negation | not | ~ | | | | | (highest precedence) |
| 2. Multiplying/shift operators | * | / | mod | & | << | >> | |
| 3. Unary sign operators | + | - | | | | | |
| 4. Adding/bitwise operators | + | - | | ^ | | | |
| 5. Relational operators | = | != | < | <= | > | >= | |
| 6. Logical/conditional operators | and | or | ?: | | | | (lowest precedence) |

Expressions

For binary operators, in general both operands must have the same type except that we can mix types `Integer` and `Byte` since `Byte` values are promoted to type `Integer` when used in expressions. Objects are considered to have the same type if and only if they have the same type name or they are declared using identical array type constructors. Thus, two distinct type definitions are considered different even though they may be structurally identical. This is referred to as “name equivalence” of types.

Example 1

```
var x : array[10] of Integer;
var y : array[10] of Integer;
```

In this example, `x` and `y` have the same type, “`array[10] of Integer`”.

Example 2

```
type T1 = array[10] of Integer;
type T2 = array[10] of Integer;

var x : T1;
var y : T1;
var z : T2;
```

In this example, `x` and `y` have the same type, but `x` and `z` do not even though they have the same structure.

Logical expressions (expressions involving logical operators “`and`” or “`or`”) use short-circuit evaluation. For example, given an expression of the form “`expr1 and expr2`”, the left operand (`expr1`) is evaluated first. If the result is false, the right operand (`expr2`) is not evaluated, and the truth value for the compound expression is false.

C.5 Statements

Assignment Statement

The assignment symbol is “:=”. An assignment statement has the following form.

```
variable := expression;
```

Example 1

```
i := 2*i + 5;
```

The variable on the left and the expression on the right must have assignment compatible types. In general, CPRL uses named type equivalence, which means that two types are assignment compatible only if they share the same type name. Strict type equality is relaxed slightly for numeric types to allow the assignment of Byte expressions to Integer variables, and vice versa. Two composite types with identical structure but different type names are not assignment compatible.

Example 2

```
type T1 = array[10] of Integer;
type T2 = array[10] of Integer;

var x : T1;
var y : T1;
var z : T2;
...
x := y;    // allowed
x := z;    // *** Illegal in CPRL ***
```

String literals, and constants representing string literals, can be assigned to a variable of type String.

Example 3

```
var emailAddr1, emailAddr2 : String;
...
emailAddr1 := "natem@gmail.com";
emailAddr2 := "john_smith@hamptoncommunitycollege.edu";
```

Compound Statement

A compound statement is a sequence of zero or more statements enclosed in braces “{” and “}”. A compound statement can be used anywhere a single statement can be used. Compound statements are commonly used in conjunction with if statements and loop statements. For example, the body of a loop is exactly one statement, but it may be a compound statement.

Example

```

while i < length loop
{
    writeln a[i];
    i := i + 1;
}

```

} compound statement

If Statement

An if statement in CPRL is similar to an if statement in Java or Kotlin except that the boolean expression is followed by the keyword “then” and is not required to be enclosed in parentheses. An if statement can contain an optional else clause.

Examples

```

if a[i] = searchValue then
    found := true;

```

```

if x >= y then
    max := x;
else
    max := y;

```

```

if x < y then
{
    // swap x and y
    temp := x;
    x := y;
    y := temp;
}

```

Loop and Exit Statements

A general loop statement consists of the keyword “loop” followed by a statement, which is often a compound statement. A loop statement may be preceded by an optional “while” clause. An exit statement can be used to exit the innermost loop that contains it. Note that an exit statement must be nested within the body of a loop statement; it cannot appear as a standalone statement outside of a loop.

Example 1.

```

while i < n loop
{
    sum := sum + a[i];
    i := i + 1;
}

```

Example 2.

```

loop
{
  read x;
  exit when x = SIGNAL;
  process(x);
}

```

A `forLoop` statement can be used to iterate over a range of integer or enum values.

Example 3.

```

for i in 1..10 loop    // i has type Integer
  writeln a[i];

```

The loop variable, `i` in this example, is implicitly declared as a variable of type `Integer` and is scoped to the loop body. The starting and ending values of the loop can be arbitrary expressions; they are not required to be constants.

Range expressions in a `forLoop` can also have an enumerated type. In the example below, loop variable `d` has type `Day`.

Example 4.

```

for d in Day.Sun..Day.Sat loop    // d has type Day
  writeln dayNames[d];

```

Range expressions can also have type `Byte`, but the loop variable in this case will have type `Integer`.

Example 5.

```

var start : Byte := 0;
var end   : Byte := 9;
for b in start..end loop    // b has type Integer
  writeln b;

```

Input/Output Statements

CPRL defines only sequential text I/O for two basic character streams – standard input and standard output. I/O is provided by `read`, `write`, and `writeln` statements. Note that `readln` is a reserved word, but it is currently not used for input in CPRL.

The `write` and `writeln` statements can have multiple expressions separated by commas. The `writeln` statement appends an end-of-line after all expressions have been written, but the `write` statement does not. Both input and output are supported for integers, characters, and strings. Output for type `Boolean` is supported, but it writes out an integer representation instead of “true” or “false”. Similarly, output for values of an enumerated type print the underlying integer value, not the name of the enum constant.

Examples

```
read x;      // assume that x has type Integer
writeln "The answer is ", 2*x + 1;
```

There is an important distinction between reading a value for an integer variable and reading a value for a byte variable. Reading a value for an integer variable will read a sequence of character digits and convert the sequence to an integer value. Reading a value for a byte variable will read a single binary byte value, not a sequence of characters representing the byte value. This difference is illustrated in the following example.

```
proc main()
{
  var n : Integer;
  var b : Byte;

  writeln "Enter a value for n.";
  read n;      // enter 123<Return>
  writeln n;    // writes 123 as expected

  writeln "Enter a value for b.";
  read b;      // enter 123<Return>
  writeln b;    // writes 49, the ASCII/Unicode value for 1
}
```

C.6 Programs

A program has an optional list of initial declarations (const declarations, var declarations, and type declarations) followed by a list of subprogram declarations. One of the subprograms must be a parameterless procedure named “main()”, which serves as the starting point for program execution.

Example

```
var x : Integer;

proc main()
{
  read x;
  writeln "x = ", x;
}
```

C.7 Subprograms

CPRL provides two separate forms of subprograms – procedures and functions. A procedure does not return a value; it is invoked through a procedure call statement. A function must return a value and is invoked as an expression. Keywords “proc” and “fun” are used to start the declarations of procedures and functions, respectively. Recursive invocations of subprograms are allowed. Subprograms are not required to be declared before they are called. All subprogram names in a program must be distinct.

Procedures

Procedures are similar to void functions in C and programming languages derived from C. Explicit return statements are allowed within the subprogram body, but unlike functions, a return statement in a procedure must **not** be followed by an expression. Procedure calls are statements.

Example 1:

```
proc writeBoolean(b : Boolean)
{
  if b then
    write "true";
  else
    write "false";
}
```

Example 2 (Quick Sort)

```
proc quickSort(var a : A, fromIndex : Integer, toIndex : Integer)
{
  var i, j, pivot, temp : Integer;

  i := fromIndex;
  j := toIndex;
  pivot := a[(fromIndex + toIndex)/2];

  // partition a[fromIndex]..a[toIndex] with pivot as the dividing item
  while i <= j loop
  {
    while a[i] < pivot loop
      i := i + 1;

    while a[j] > pivot loop
      j := j - 1;
```

```

    if i <= j then
    {
        // swap a[i] and a[j]
        temp := a[i];
        a[i] := a[j];
        a[j] := temp;

        // update i and j
        i := i + 1;
        j := j - 1;
    }

    if fromIndex < j then
        quickSort(a, fromIndex, j);    // sort top part

    if i < toIndex then
        quickSort(a, i, toIndex);      // sort bottom part
}

```

Assume that we have the following declarations.

```

const arraySize := 10;
type A = array[arraySize] of Integer;
var a : A;

```

Then we could call this procedure to sort the array as follows.

```

quickSort(a, 0, arraySize - 1);

```

Functions

Functions are similar to procedures except that functions return values. Function calls are expressions. A function returns a value by executing a “return” statement of the following form.

```

return <expression>;

```

Example 1

```

fun max(x : Integer, y : Integer) : Integer
{
    return x > y ? x : y;
}

```

Example 2

```
fun sgn(n : Integer) : Integer
{
  if n > 0 then
    return 1;
  else if n < 0 then
    return -1;
  else
    return 0;
}
```

Parameters

There are two parameter modes in CPRL – value parameters and variable parameters. Value parameters are passed by value (a.k.a. copy-in) and are the default. Variable parameters are passed by reference and must be explicitly declared using the “var” keyword. Unlike other types, arrays are always passed by reference to subprograms regardless of whether or not they are declared as variable parameters.

Example

```
proc inc(var x : Integer)
{
  x := x + 1;
}
```

Functions cannot declare variable parameters; only value parameters can be declared for functions. However, arrays are always passed by reference, even for functions, where they are not declared as var parameters.

Return Statements

A return statement terminates execution of a subprogram and returns control back to the point where the subprogram was called. A return statement within a function must be followed by an expression whose value is returned by the function. The type of the expression must be equal to the return type of the function. A return statement within a procedure must not be followed by an expression; it simply returns control to the statement following the procedure call statement.

A procedure has an implied return statement as its last statement, and therefore most procedures will not have an explicit return statement. A function requires one or more return statements to return the function value. There is no implicit return statement at the end of a function.

Built-in Subprograms

CPRL provides three built-in functions and one built-in procedure with the following signatures.

```

fun chr(n : Integer) : Char    // returns the character with the
                                // specified Unicode code point

fun eof() : Boolean            // returns true if standard input has
                                // reached the end of file

proc inc(var n : Integer)      // increments n

fun ord(c : Char) : Integer    // returns the ordinal value of a character

```

Functions `chr()` and `ord()` provide the capability to convert between CPRL types `Char` and `Integer`. For example, `ord('A')` returns the value 65, and `chr(0x20AC)` returns the Euro sign '€'. When calling `chr()`, there is no checking that the integer parameter is a valid Unicode code point, but if it is, the two functions are inverses of each other; i.e., if `c` has type `Char` and `n` has type `Integer` (and if `n` is a valid Unicode code point) then `chr(ord(c)) = c` and `ord(chr(n)) = n`.

Procedure `inc()` simply increments its parameter. If `n` is an integer variable, writing `inc(n)` in CPRL is essentially equivalent to writing `++n` as a statement (i.e., on a line by itself followed by a semicolon) in C-based languages.

Function `eof()` signals end-of-file from standard input. Reading a file using the input redirection operator `<` works as expected, but the function does not detect end of file until a read is attempted. Following is an example function that reads and echoes the bytes in a binary file.

```

proc main()
{
  var b : Byte;
  loop
  {
    read b;
    exit when eof();
    writeln b;
  }
}

```

Assuming that the above program is in a CPRL source code file named `ReadBytes.cprl`, you could run the following to echo the bytes in the object code file.

```

cprl ReadBytes < ReadBytes.obj

```